Laboratorio: Análisis de complejidad

Teoría de la Computación
Problemas resueltos: 2 (25%), 3 (50%) y 4 (25%)

Problema 2

```
1  void function(int n) {
2      if (n <= 1) return;
3      int i, j;
4      for (i = 1; i <= n; i++) {
5          for (j = 1; j <= n; j++) {
6              printf("Sequence\n");
7              break;
8          }
9      }
10 }
```

Parte a

Caso $n \leq 1$ (línea 2):

$$T(n) = c_1 = O(1)$$

Caso $n \geq 2$:

Ciclo externo (línea 4): $i = 1, 2, \ldots, n \;\Rightarrow\; n$ iteraciones.

Ciclo interno (líneas 5–7): $j = 1$, $1 \leq n$ verdadero, `printf`, `break` $\Rightarrow$ 1 iteración por cada i.

$$\begin{aligned} T(n) &= c_1 + \sum_{i=1}^{n} \left(c_2 + \sum_{j=1}^{1} c_3 \right) \\ &= c_1 + \sum_{i=1}^{n} (c_2 + c_3) \\ &= c_1 + (c_2 + c_3)\, n \\ &= a\, n + b, \qquad a = c_2 + c_3, \quad b = c_1 \end{aligned}$$

Número de impresiones:

$$\sum_{i=1}^{n} 1 = n$$

Cota superior, para $n \geq 1$:

$$\begin{aligned} a\, n + b &\leq a\, n + b\, n = (a + b)\, n \\ T(n) &\leq c\, n, \qquad c = a + b, \quad n_0 = 1 \end{aligned}$$

$$\boxed{T(n) = O(n)}$$

Parte b

n	Impresiones	Tiempo (s)
1	0	0.000000024
10	10	0.000000255
100	100	0.000002405
1 000	1 000	0.000023437
10 000	10 000	0.000230604
100 000	100 000	0.002340159
1 000 000	1 000 000	0.022874700

Tabla 1: Problema 2: tamaño de input vs. tiempo

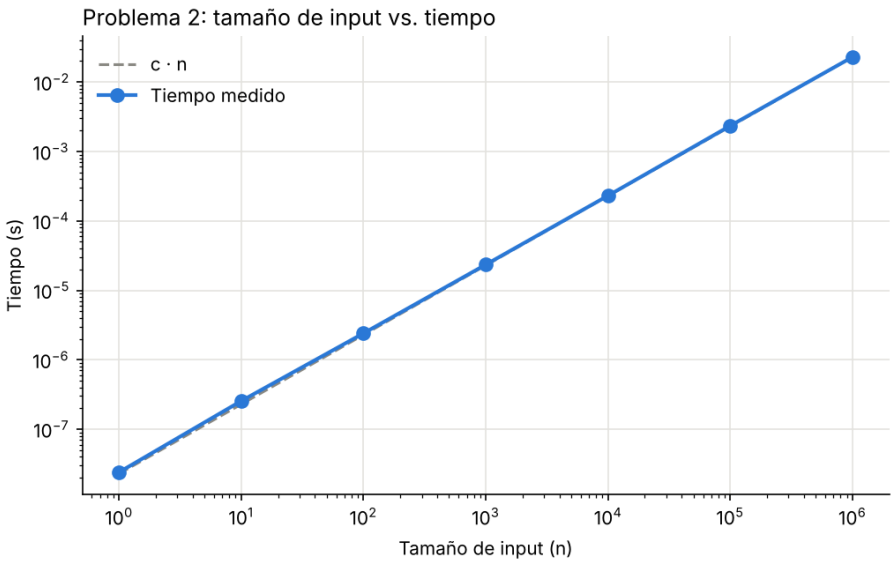


Figura 1: Problema 2: tamaño de input vs. tiempo (escala log-log)

Problema 3

```
1 void function(int n) {
2     int i, j;
3     for (i = 1; i <= n/3; i++) {
4         for (j = 1; j <= n; j += 4) {
5             printf("Sequence\n");
6         }
7     }
8 }
```

Parte a

Ciclo externo (línea 3): $i = 1, 2, \ldots, \lfloor n/3 \rfloor \;\Rightarrow\; \lfloor n/3 \rfloor$ iteraciones.

Ciclo interno (línea 4): $j = 1, 5, 9, \ldots, 1 + 4(k-1)$

$$\begin{aligned} 1 + 4(k-1) &\leq n \\ 4(k-1) &\leq n - 1 \\ k &\leq \frac{n-1}{4} + 1 = \frac{n+3}{4} \\ k &= \left\lfloor \frac{n+3}{4} \right\rfloor = \left\lceil \frac{n}{4} \right\rceil \end{aligned}$$

$$\begin{aligned} T(n) &= \sum_{i=1}^{\lfloor n/3 \rfloor} \sum_{k=1}^{\lceil n/4 \rceil} c \\ &= \sum_{i=1}^{\lfloor n/3 \rfloor} c \left\lceil \frac{n}{4} \right\rceil \\ &= c \left\lfloor \frac{n}{3} \right\rfloor \left\lceil \frac{n}{4} \right\rceil \end{aligned}$$

Cota superior, para $n \geq 1$:

$$\begin{aligned} T(n) &\leq c \cdot \frac{n}{3} \left(\frac{n}{4} + 1 \right) \\ &= c \left(\frac{n^2}{12} + \frac{n}{3} \right) \\ &\leq c \left(\frac{n^2}{12} + \frac{n^2}{3} \right) \\ &= \frac{5c}{12}\, n^2 \end{aligned}$$

$$T(n) \leq c'\, n^2, \qquad c' = \frac{5c}{12}, \quad n_0 = 1$$

$$\boxed{T(n) = O(n^2)}$$

Parte b

n	Impresiones $\lfloor n/3 \rfloor \lceil n/4 \rceil$	Tiempo (s)
1	0	0.000000025
10	9	0.000000249
100	825	0.000019690
1 000	83 250	0.001884230
10 000	8 332 500	0.191393900
100 000	833 325 000	26.767999600
1 000 000	83 333 250 000	4049.630935200

Tabla 2: Problema 3: tamaño de input vs. tiempo

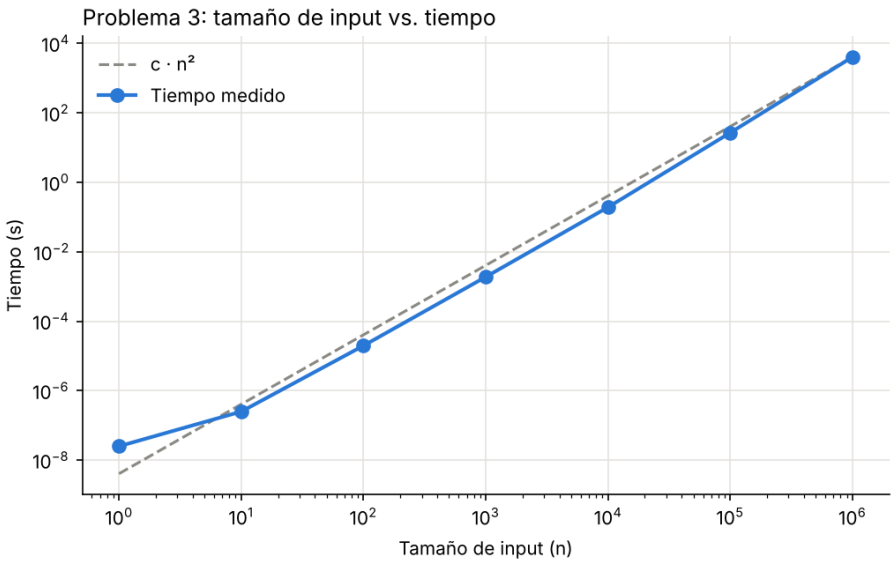


Figura 2: Problema 3: tamaño de input vs. tiempo (escala log-log).

Problema 4

Operación básica: comparación entre elementos. Tamaño del arreglo: n.

Linear Search

Mejor caso (el elemento está en la posición 1):

$$T(n) = 1 \;\Rightarrow\; O(1)$$

Peor caso (el elemento está en la posición n o no está):

$$T(n) = \sum_{i=1}^{n} 1 = n \;\Rightarrow\; O(n)$$

Caso promedio (el elemento está en la posición i con probabilidad $1/n$, costo i):

$$\begin{aligned} T(n) &= \sum_{i=1}^{n} i \cdot \frac{1}{n} \\ &= \frac{1}{n} \cdot \frac{n(n+1)}{2} \\ &= \frac{n+1}{2} \;\Rightarrow\; O(n) \end{aligned}$$

Binary Search

Mejor caso (el elemento está en la posición central):

$$T(n) = 1 \;\Rightarrow\; O(1)$$

Peor caso (el elemento no está o se encuentra en la última división):

$$\begin{aligned} T(n) &= T\left(\frac{n}{2}\right) + 1, \qquad T(1) = 1 \\ &= T\left(\frac{n}{4}\right) + 2 \\ &= T\left(\frac{n}{8}\right) + 3 \\ &\vdots \\ &= T\left(\frac{n}{2^k}\right) + k \end{aligned}$$

$$\frac{n}{2^k} = 1 \;\Rightarrow\; k = \log_2 n$$

$$T(n) = T(1) + \log_2 n = \log_2 n + 1 \;\Rightarrow\; O(\log n)$$

Caso promedio ($n = 2^k - 1$, búsqueda exitosa, cada elemento con probabilidad $1/n$; hay 2^{i-1} elementos que se encuentran con i comparaciones):

$$\begin{aligned} T(n) &= \frac{1}{n} \sum_{i=1}^{k} i \cdot 2^{i-1} \\ &= \frac{1}{n} \left[(k-1)\, 2^k + 1 \right] \\ &= \frac{(k-1)(n+1)+1}{n} \\ &= (k-1) + \frac{k}{n} \\ &= \log_2(n+1) - 1 + \frac{\log_2(n+1)}{n} \;\Rightarrow\; O(\log n) \end{aligned}$$

Quick Sort

Costo de la partición de n elementos: $c\,n$.

Mejor caso (el pivote divide el arreglo en dos mitades):

$$\begin{aligned} T(n) &= 2\,T\Big(\frac{n}{2}\Big) + c\,n, \qquad T(1) = c \\ &= 4\,T\Big(\frac{n}{4}\Big) + 2\,c\,n \\ &= 8\,T\Big(\frac{n}{8}\Big) + 3\,c\,n \\ &\vdots \\ &= 2^k\,T\Big(\frac{n}{2^k}\Big) + k\,c\,n \end{aligned}$$

$$\frac{n}{2^k} = 1 \;\Rightarrow\; k = \log_2 n$$

$$T(n) = n\,T(1) + c\,n\log_2 n = c\,n + c\,n\log_2 n \;\Rightarrow\; O(n \log n)$$

Peor caso (el pivote es siempre el mínimo o el máximo, por ejemplo arreglo ya ordenado con pivote en un extremo):

$$\begin{aligned} T(n) &= T(n-1) + c\,n, \qquad T(1) = c \\ &= T(n-2) + c\,(n-1) + c\,n \\ &= T(n-3) + c\,(n-2) + c\,(n-1) + c\,n \\ &\vdots \\ &= T(1) + c \sum_{i=2}^{n} i \\ &= c \sum_{i=1}^{n} i \\ &= c\,\frac{n(n+1)}{2} \;\Rightarrow\; O(n^2) \end{aligned}$$

Caso promedio (el pivote queda en cada posición con probabilidad $1/n$; partición con $n-1$ comparaciones; $T(0) = T(1) = 0$):

$$\begin{aligned}
T(n) &= (n-1) + \frac{1}{n}\sum_{k=0}^{n-1}[T(k) + T(n-1-k)] \\
&= (n-1) + \frac{2}{n}\sum_{k=0}^{n-1} T(k) \\
n\,T(n) &= n(n-1) + 2\sum_{k=0}^{n-1} T(k) \\
(n-1)\,T(n-1) &= (n-1)(n-2) + 2\sum_{k=0}^{n-2} T(k) \\
n\,T(n) - (n-1)\,T(n-1) &= 2(n-1) + 2\,T(n-1) \\
n\,T(n) &= (n+1)\,T(n-1) + 2(n-1) \\
\frac{T(n)}{n+1} &= \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)}
\end{aligned}$$

$$\begin{aligned}
\frac{T(n)}{n+1} &\leq \frac{T(n-1)}{n} + \frac{2}{n+1} \\
&\leq \frac{T(1)}{2} + \sum_{k=2}^{n} \frac{2}{k+1} \\
&= 2\sum_{k=3}^{n+1} \frac{1}{k} \\
&\leq 2\int_{2}^{n+1} \frac{dx}{x} \\
&= 2\ln\frac{n+1}{2} \\
T(n) &\leq 2\,(n+1)\ln(n+1) \;\Rightarrow\; O(n\log n)
\end{aligned}$$

Resumen

Algoritmo	Mejor caso	Caso promedio	Peor caso
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$